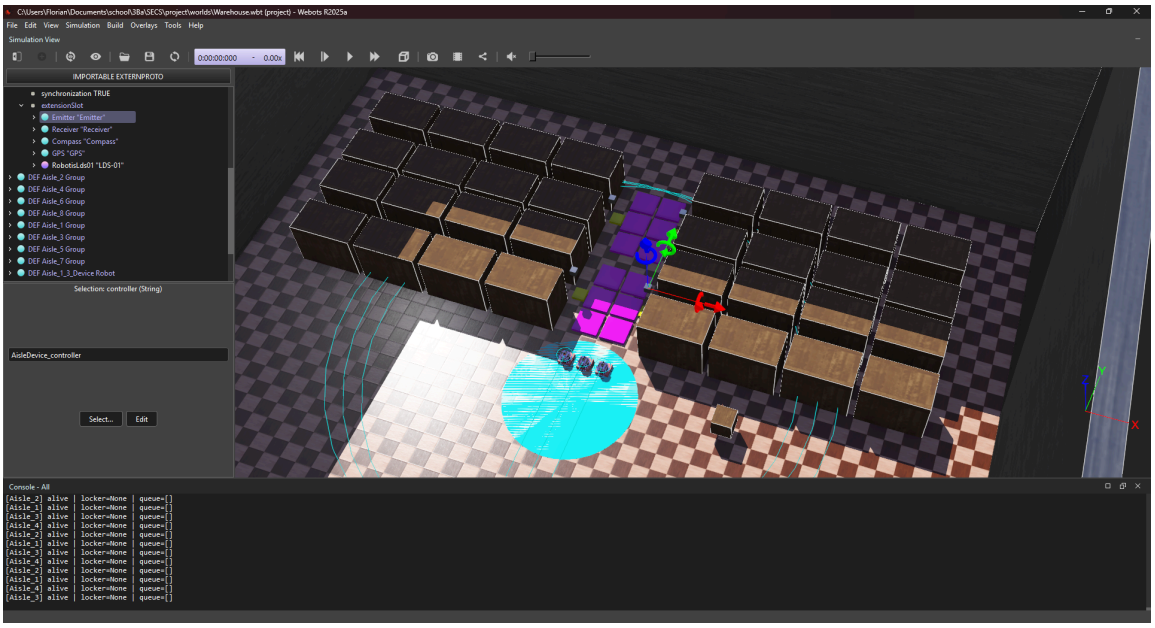
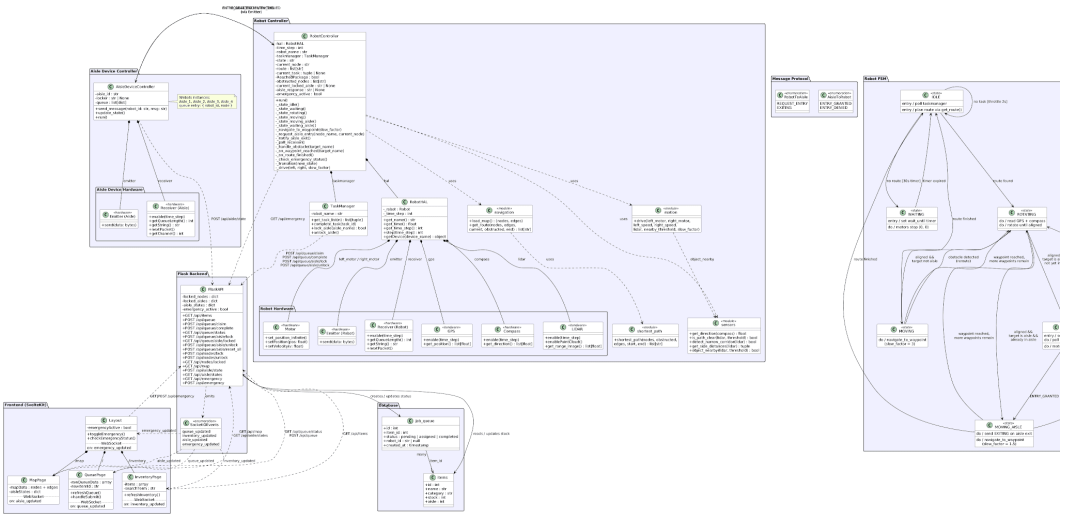
Rapport
Software engineering for cyber-physical systems





Salaets Florian
Nysmans Tobe
Mertens Lars

Table of contents

Table of contents **2**
1 Warehouse architecture **3**
Nodes 3
2 The robot **5**
Final State Machine 5
Moving 5
Collision prevention 6
Hardware abstraction layer 6
Aisle devices 6
Emergency state 7
3 The backend **7**
Endpoints 7
Concurrent tasks 8
Collisions in DB 8
4 The Database **8**
5 The webinterface **8**
6 Concurrency **10**
7 Testing **10**
8 Deployment **11**
9 Running the project **11**
Annex A: Plant UML (project elements) **12**
Annex B: Plant UML (diagrams) **15**

1 Warehouse architecture

First and foremost is the environment we created for our bots to operate in. We created a world in Webots with boxes grouped together to form aisles. We deliberately left gaps in between shelves to make the traversal through these aisles harder for our bots. 2 aisles together are accessed through the same path/hallway and can only be entered by one bot. Next we added a drop off zone, visualized by a blue cylinder which has no collision properties The actual drop off points are envisioned by light pink squares. Nodes are added to the hallway of the warehouse, as most movement of the bots will happen there. This creates some structure in the movement of the bots. These nodes are visualized by magenta squares. The nodes that represent an aisle entrance are visualized by a yellow square. This can be seen in figure 1 below.

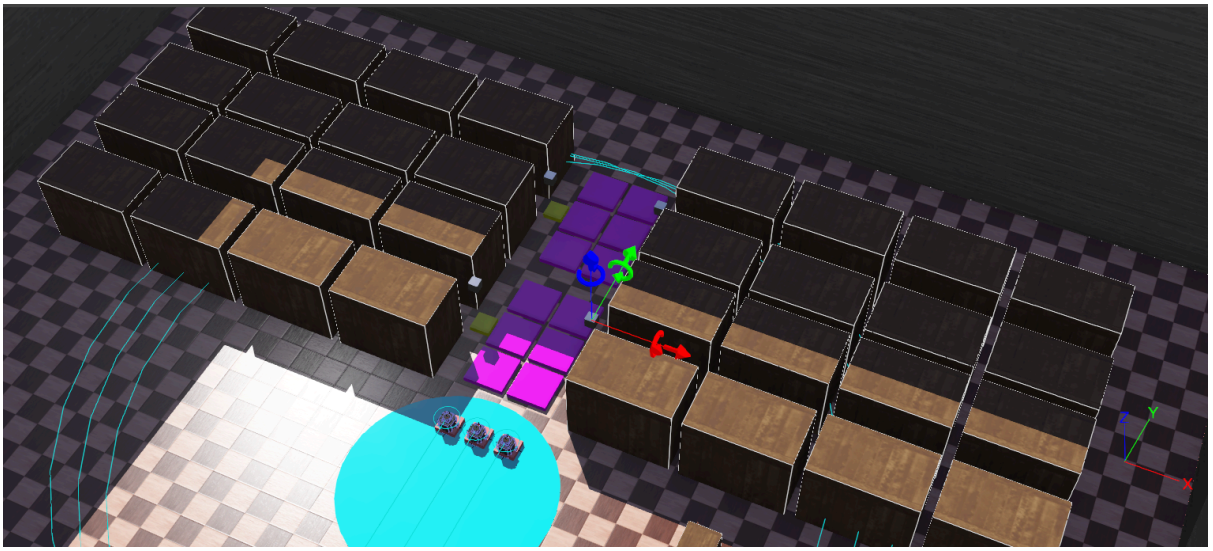


Figure 1: Screenshot of the warehouse in Webots

The metallic floating boxes that can be seen in the figure above are aisle devices, these will be talked about later in this report.

Nodes

The bots navigate through a web of nodes, connected with directional edges in between. We opted for a technique like this to make bots move in a more structured way and to make collision prevention easier. This means that setting our bots up in an actual warehouse means that one would need to take some time to configure a map with these nodes and their locations. We acknowledge this extra work, but we think that the benefits outclass this one-time effort.

Robots use a breadth-first search to find the shortest path to its destination. This means that it checks its neighbour node., If they are in the visited list they get skipped, because there was already a shorter path found, else they get added to the list together with the path to reach that node. This then is repeated for neighbours of the nodes that weren't in the list already until the destination is found. This algorithm can be found in the file

“shortest_path.py” under “controllers/robot_controller”. The robots have access to a map that gives the layout of the warehouse.

Earlier it was mentioned that the edges are directional. We opted for this to implement some traffic rules in the warehouse. Our warehouse has a 2x6 grid which is shown in figure 2.a, a different possible grid is shown in figure 2.b.

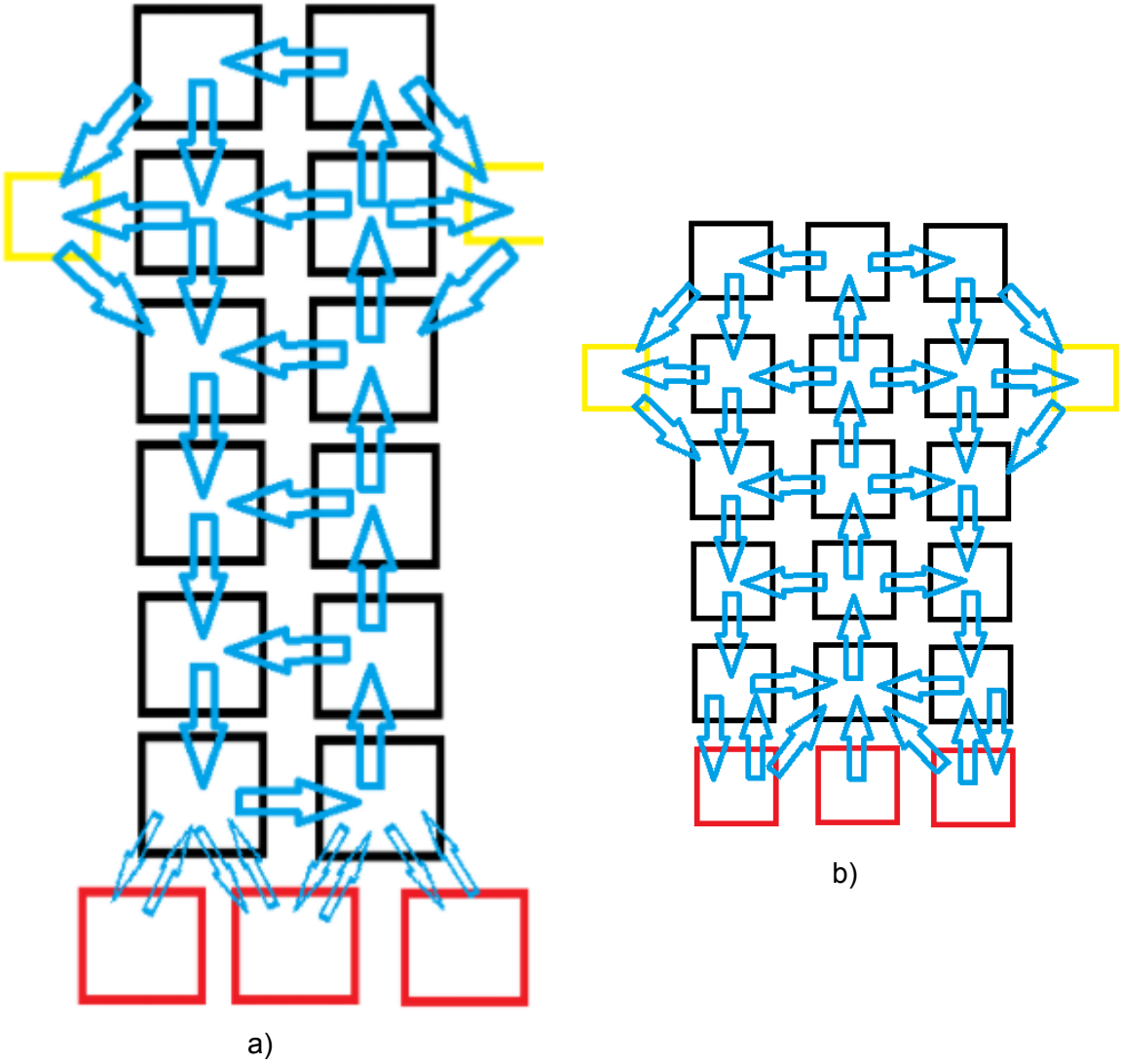


Figure 2: Nodes and edges of a) our warehouse, b) other possible layout

The figure shows that the possible paths are chosen carefully. It is impossible to get a frontal crash between two robots. On the other hand, it is likely that a robot is following another robot through the warehouse. In that case, the chasing bot should slow down or stop for a moment to prevent a collision. In case robots manage to get too close to each other, they go back to their latest visited node and reroute. But that logic is explained in another section of this report.

2 The robot

Our bots are an extension of the TurtleBot Burger Robot that is included with the default assets in Webots. We added a couple of devices to the bot:

- A GPS to locate itself in the warehouse
- A compass to get the direction the direction the bot is looking to
- A Lidar to scan the surroundings
- An emitter and receiver to send and receive messages from the aisle devices (explained later in this report)

Final State Machine

The bots operate using a Final State Machine or FSM. A bot has a (initial) state, does actions belonging to that state and then checks the conditions to switch their state. Figure 3 gives the 6 states of the FSM and when they change. The link to the Plant UML can be found in annex B.

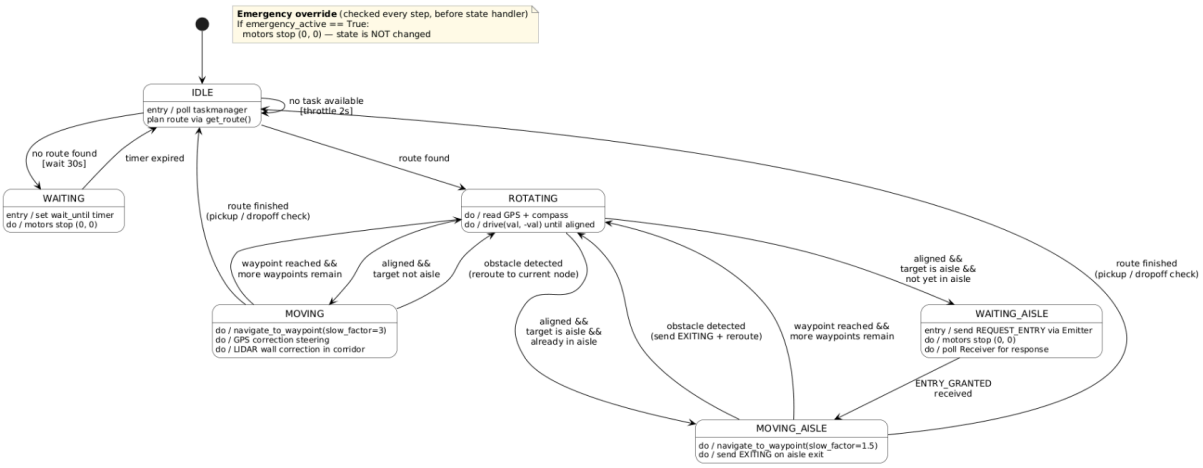


Figure 3: FSM states and changes.

Moving

The robot always rotates before moving towards the next waypoint, but also corrects itself during the traversal. If the angle difference between the bot and the target becomes too big, the robot corrects itself. In aisles, the robot corrects itself using the lidar. If the distance between the robot and the shelves becomes too narrow, the bot corrects itself.

The calculations to find the angle, distance to nodes are calculated using the coordinates of the bot's GPS and the coordinates of the node, listed in the map.json file. Before entering an aisle, the bot first needs permission. This permission is sent by an aisle device (see below).

Collision prevention

The bot constantly checks its surroundings using the lidar. If it detects that something is close, it reduces its speed. This solved most of the collision problems, especially in combination with the routing (see nodes). In the rare case that the obstruction is not resolved, the bot stops moving and waits further to see if the obstruction becomes resolved. After some time, the bot moves back to his last node and tries to find an alternative route.

Hardware abstraction layer

The hardware abstraction layer (HAL) serves as an intermediary between the control logic and the underlying physical or simulated hardware. In this project, the HAL decouples our robot's intelligent systems from the specific APIs of the Webots environment.

We want to avoid hardcoding our software to a specific simulator. If the control logic calls a Webots specific function directly, the code becomes dependent on the simulation. If we were to deploy that same code onto a physical bot, it would fail immediately because the real hardware uses different drivers.

By implementing a HAL, we achieve:

- Hardware independence: the main logic interacts with "Motors" and "Sensors" defined by the HAL and is not dependant on the simulator's internal objects
- portability: To move from simulation to real-world robot, we need to swap the HAL implementation. We don't have to change anything about the navigation and logic.

We implemented our HAL in 3 different parts. We have a file that contains the hardware components. Instead of dealing with raw data streams we wrap devices into classes. Features like enabling a sensor and setting its time steps are handled during initialization, reducing boilerplate code in the main controller file.

There are also 2 specialized files to define how specific robots are constructed from these components. 1 file Manages a complex suite of hardware including motors, GPS and lidar. It includes a factory function to simplify the setup process in the main script. The other file is simplified for our aisle devices that only require communication capabilities.

When looking at the UML file you can see the use of a hardware abstraction layer. Looking at the RobotController class you can see it only communicates with RobotHAL and never directly with any hardware components. Instead RobotHAL functions as a gateway to the hardware.

Aisle devices

Each entrance to an aisle has a device installed that only has an emitter and receiver to connect with nearby bots and a (wired) connection to the internet or backend. The device is there to lock aisles so only one bot can enter.

The communication between the bot and the device is as follows: The bot requests to enter an aisle and provides the node he is currently on. The device knows the status of the aisle (full/empty and the bot that's inside).

- In case the aisle was full, the device tells the bot to wait, sends its location to the backend and adds the bot to the queue of waiting bots.
- In case the aisle is empty, the device tells the bot to answer, and updates the status of the aisle (together with the id of the bot that locked it) to the backend. The device now locks the aisle.

When the bot leaves the aisle, it sends an exit message to the device. The device unlocks the aisle if no other bots are waiting or otherwise will send a message to one of the bots that's waiting. Finally, it will post the status change to the backend.

As discussed below, the backend and webapp are connected with a websocket. Meaning that all these events can be followed live in the user interface.

In the figure below, a simplified UML diagram is given for the aisle device. The robotcontroller and backend is simplified so that only the details that are of importance for the aisle device are given. The link of the Plant UML can be found in annex A.

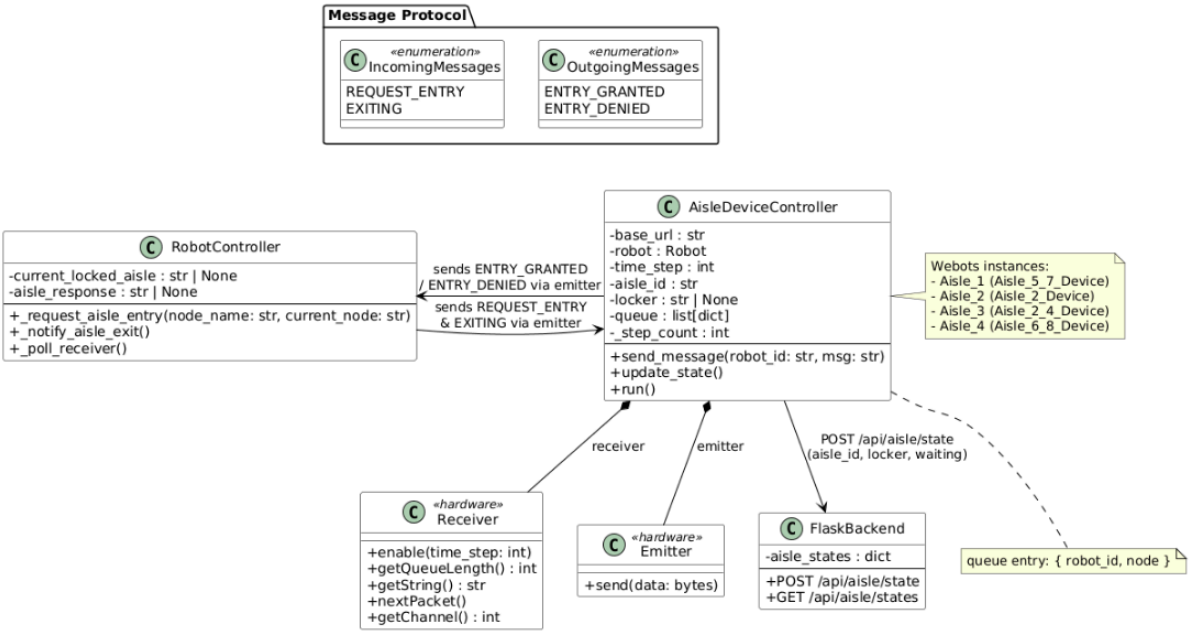


Figure 4: Plant UML of the aisle device

Emergency state

The emergency stop is a safety mechanism designed to bring all autonomous warehouse robots to an immediate stop. The system utilizes three layers of software: frontend, backend and the robot controllers.

The frontend features a big red button, that is always visible, integrated into the dashboard. In its idle state, the button with the label “EMERGENCY STOP”. The backend is a central control point that tracks the emergency status via a boolean variable. When an operator

triggers the button, the frontend sends a Post request. The backend then toggles the status and broadcasts the update to all connected clients via WebSockets.

The robots continuously monitor the emergency status by polling the backend every 0.5 seconds. Upon detecting an active emergency state, the robots immediately halt their motors. Normal operations don't resume for as long as the emergency status is active. The dual communication by polling and websockets ensures that no robot misses a status update.

3 The backend

The backend is a FLASK-server to define all our API endpoints. The backend is connected to the frontend through the use of websockets for immediate changes in the browser. We used websockets because when for example adding an item or updating the assigned bot for an item we do not want the user to have to manually reload the web browser. The backend contains all the endpoints to our database.

The webapp, bots and aisledevices all communicate with the backend to get information on aisles, tasks, locations and inventory. Though it's the central point of communication, bots can only communicate with it when in the drop off zone. We assume that the connection between the backend and aisle devices is always established.

Endpoints

- GET /api/items: Returns a list of all items in the warehouse (inventory)
- POST /api/queue: Add a new item to be retrieved to the queue
- POST /api/queue/claim: Request a new item to retrieve
- POST /api/queue/complete: Mark an item as retrieved
- GET /api/map: Returns the map of the warehouse
- POST /api/aisle/state: Update the state of an aisle - (un)locking
- GET /api/aisle/states: Get the current state of all aisles
- GET /api/emergency: Get emergency status
- POST /api/emergency: Change the emergency status

Concurrent tasks

The backend tries to send tasks that involve an aisle visit no other bots are concurrently visiting. This is better explained in the concurrency section.

Collisions in DB

When we use an API to access our database we might get trouble when 2 or 3 robots are asking for a task at the same time. We fixed this by using a try-catch block where we immediately use an atomic update so it locks the row immediately, then tries to process it.

After processing if there is a problem we just do a rollback, or otherwise we just finish the operation.

4 The Database

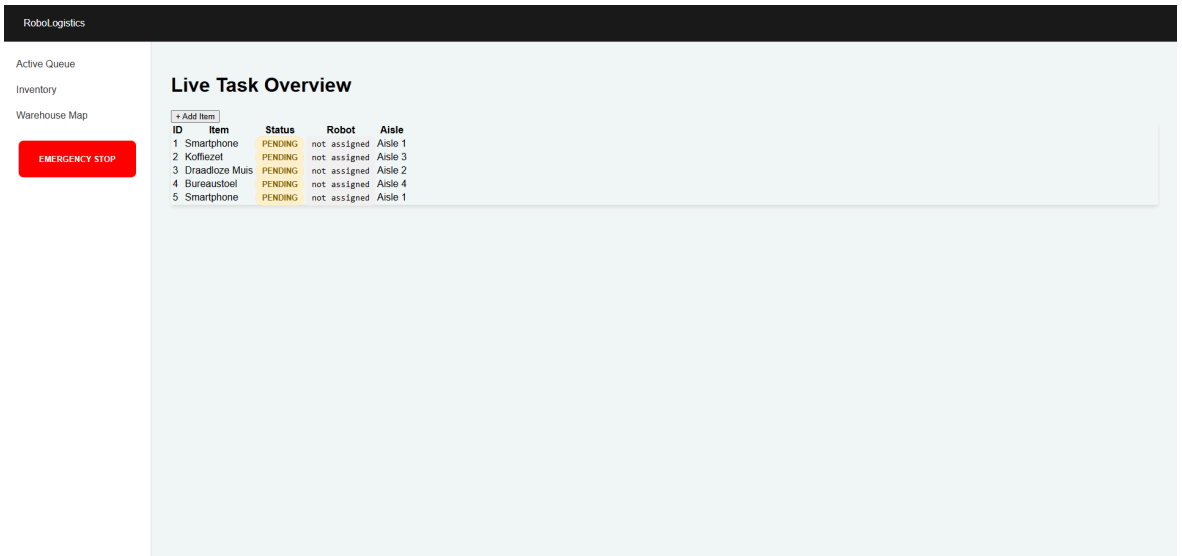
We decided to use a SQL-database because the elements we needed in the database are relational, so SQL seemed like the best option because it is a relational database with tables so it was easy to just have 2 tables. We have one table for items and one for the queue. The items have an id, name, category, stock ,and what aisle it is located in. The aisles in the database do not have the same name as the ones in Webots so we use a small dictionary to translate the aisle names from the web application to the names in Webots.

5 The webinterface

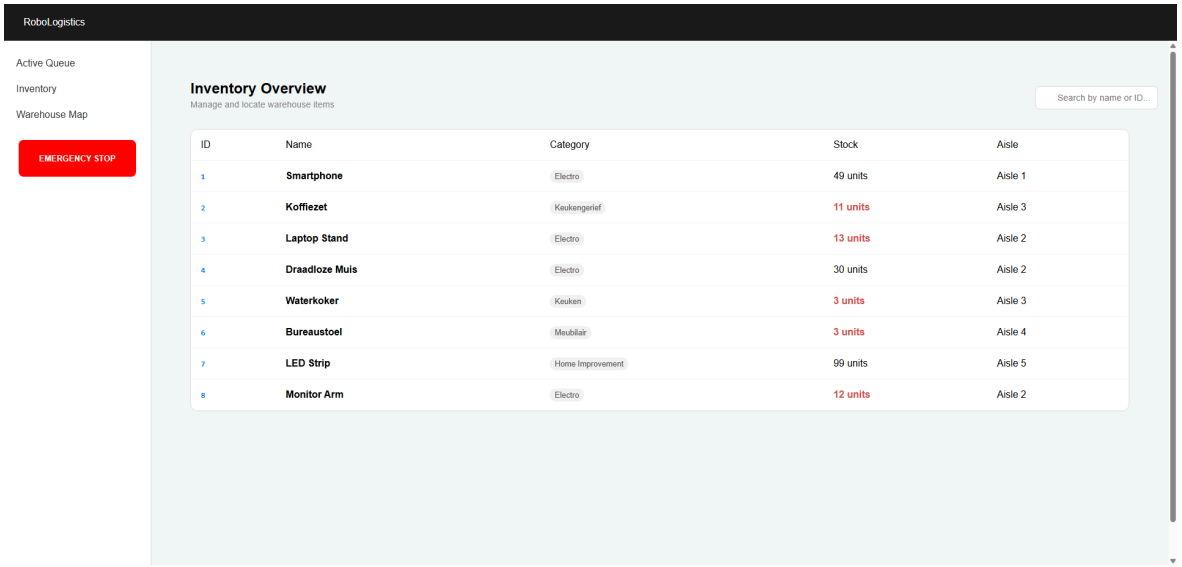
We implemented a user interface using svelte. On the website you have 3 pages:

- Active queue: an overview of the current tasks with their status and assigned bot. an option to add an item to the queue. (figure 5.a)
- Inventory: an overview of all the items in the warehouse, together with their category, id, location and stock. (figure 5.b)
- Warehouse map: an overview of the current situation in the warehouse. The status of aisles is given together with the bot that's inside. Robots emit their location when waiting for the aisle to open (as it's near an aisle device), when inside the aisle and when at the drop off point. (figure 5.c)

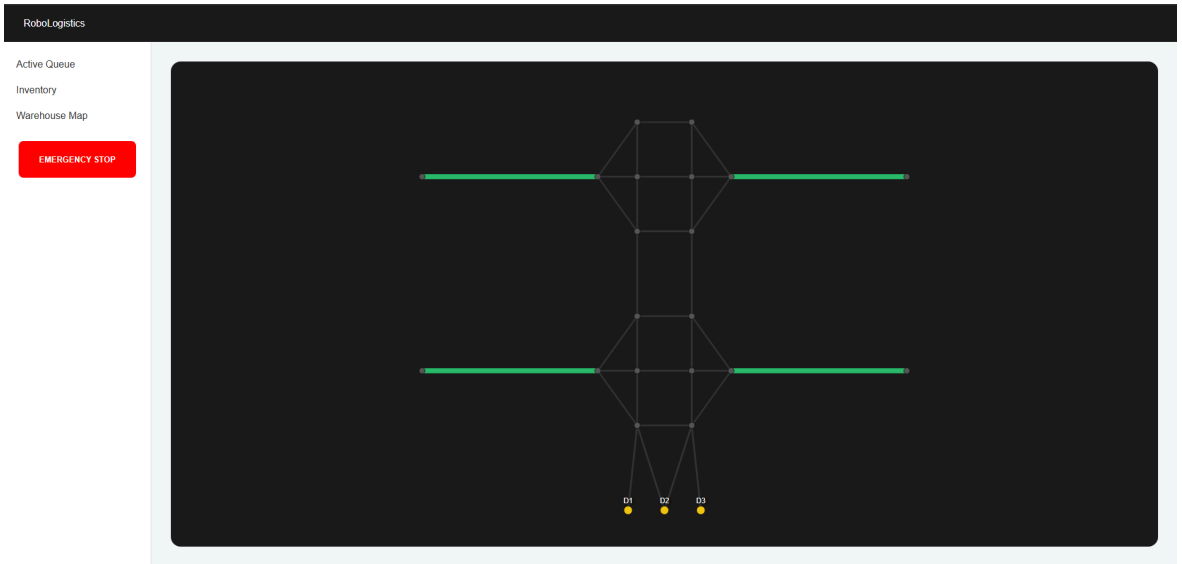
The web-interface also includes an emergency button that when clicked stops all the bots, so people can access the warehouse.



a)



b)



c)

Figure 5: Webapp a) active queue, b) inventory, c) warehouse map

6 Concurrency

Concurrency is very important to achieve a more effective system. As previously mentioned, we implemented a node system for navigation. Mainly because we don't want our bots running into each other, which would mean we would need to either wait or reroute. Both are negatively impacting efficiency. First we focus on preventing a collision, then we focus on resolving the blockage.

Another way we try to improve the efficiency is to assign a task that involves going to an aisle no other bot should be entering. For example, 2 items need to be retrieved from aisle 1, 1 item needs to be retrieved from aisle 4. We give 1 of the items to one bot, but then prefer

to give the item from aisle 4 to the other bot. This way we send both bots to different aisles, which makes that one bot won't have to wait on the other while that bot is busy in the aisle. The backend api saves what aisle is already occupied in memory, meaning that a restart of the backend (due to a crash or some other problem) will lose this efficiency boost temporarily.

7 Testing

Some of the smaller stuff, like the shortest path algorithm, was tested with a written test. If you run the "shortest_path.py" file separately, the code is tested with the data of the map provided by a .json-file.

But most of our testing happened by non-written edge-to-edge tests. This means we implemented something, then checked in the Webots world if it worked. This meant we were relying on older working parts of the project, which definitely isn't ideal. Though it is really easy to spot if a bot isn't doing what it is supposed to do. Other things, like driving through an aisle, were tested by placing the bot in a dummy world and observing.
Another way we tested was by adding a lot of debug messages. Almost every action (like switching states, completing a route, retrieving tasks, ….) has a print message in the console. This helped us identify, but foremost solve, almost every problem.

8 Deployment

How the final project is deployed is given below in figure 6. The Plant UML link can be found in annex B.

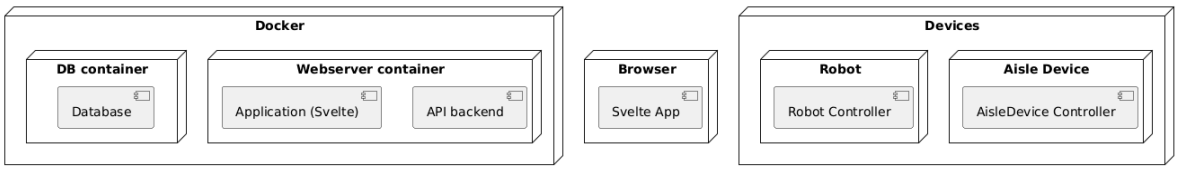


Figure 6: Deployment diagram

The database, api and application (backend) are running on a server inside a docker container. The app is served through an internet browser. The controllers are deployed on the robots themselves.

9 Running the project

Setting up the project can be done with these simple steps:

1) Unpack the project or clone from github.
2) Get all imports with pip using the requirements.txt in the project root. Make sure to use the python version that Webots is using. Use the following command (in project root): $ py -m pip install -r requirements.txt

3) Start the backend api and serve the webapp by using the following command (in project root): $ docker-compose up –build. Docker desktop is required to be installed for this.
4) Open the warehouse.wbt file in Webots. The simulation should be ready to run.
5) Visit the user interface in your browser http://localhost:5173

Change the python version Webots is using by editing “Python Command” under tools → preferences in Webots (if the versions don’t match).

Since the file is too big, a demo is uploaded to youtube and can be found by following this link: https://youtu.be/rM-I8zwOObc.

Annex A: Plant UML (project elements)

Plant UML link for robot:
https://editor.plantuml.com/uml/fLTjRzis4FvkmF-1w3xCMUDcduKYQ9guQO0cTHtI3MqXqE9POYsH7aavzRRyznr9KM-Mis9jWCgwUtXtFDuzHxtIXYfJ5ZbPo9Kqbr8O9VCSr7Wq7kaD5rkgQ46Id6fzOOpYgzB0JIB5FVyBoCy76FBtU4JmtskQR58bIy4kPIuL-ItZ1YgLL0oK5x_E4UV55qh9XuxqiMrzYoAQmZ7xFxpv6V_U7FNmQFVZ8-njD5YSaepc9FBgjnVlWjJm0c9jO8iwBamGAmkB1Iq0vTge6axrfg02GrMeMEBRhNyB04ouwQr9IgL0e3N9UXhSeGFdN9kFAF_SNsBTeTwKsnp8F-HE2WYG1T0a0pQl4XUHbPHvKCeLMYiJ0ynvrK_vo6MoGHpbEg_2wtbocbY1tagXXo5GW4f19FkO9eRlUk5C_VCvKQMOd8GriKjLp5aE1y87oWqNwO5SINmCAGgvEoxsUplcv40hw8wd5c0aOlPRYNKnqRbyYDUuEwaQf88_Iz36MuWncMe_iVbsXNFQEVYMUMduUX_MVEMcvNgBDOjM4y0SgfOyemBJ5DjZfGa6YrsTWg_F1YH57IxQSDLn5EWgBrvpmRM5DJfSbMpYvZnjgahT0XX5XSR4IJ4HyE1JsMYPmiWdEQpDAL4ypV0ncBc4vabCHOgROdozdcobjtbAgeXHq4CpB60g4ZY6xNC0DddJ_B5hwD3_pMO6kx-krv2woKcxRxtSBdNoTIwfwMkSKQUkpJudJbPhUqfSUGKxZljZxk7F335ozGKIK-qlyEIpjrIn1whWMTYZt_dr_9wyV9bLolFpeFK-GD2LhPuwXBQoogG_sfF066vt5MUu21fVbxBOskTt-6CSY_sR7RwxkRfO_5ztNZcttN6PoxAk-IOOBEmK-QZ0_7vJEBSII_g9S7GheJ4NjjaCTDrQo0wGWRdPJtPLBVKHyt0eQEm8vDSZDdeHpmfk33mTis0JHWr50jyRqCDs5XL3VSVHVoYXX7SWKfDrswH6tEDrn59wk_MyKi1NO-SUrDJvs6b_t8IMoXPc8LcPjsBhLrtYI_YWZRcEjzHaCN8jLPESCwgGRJASWPdCsKbdI3qd30pQmuQrTvmuaKfn9gjbVQprhpc3civqWtDZFW1zsyS20rZjdmYXjUz2saqDRhlXPPo9MBOLEuSNfz1R07RwINYDVoELOnz6NFllqtehYcfy7ei4qypYWdfoTDDIdn9WQLrhLJFQ0TLHr_Fqj7MzGPrW9zthJRiICgcCdT3sD0V3wI22luvzxzjTfF_3M-iws4oU8pV9peonrxpOsfo8shehXiy1PGir1tidGUZ4uJdhvlqD_h-n5sUCiNPVkxcUBSaPtVApUhGVQAnL7QJpz_SjyLdmtnz3jw2r_J77cvbCPDuRHpS25s8NLp3RaY1AZC3LHvSpPdVBnH_nzUBYRZcxweklPdStGVhOzI2-DAdyHW-BsOTVPlVBs9biUVZzPdbpTpt491VsakR7SlUJmc5-T2FGiJD41nyUq_FMnqPV-TEKoGSnRMugaVq-yKRjwmeo06L8eDVAWbM5fNRKHoJTwW5T6CqHgHXk0EEdQUGxV43Inrj46j9u0jHYam5Kc3iHVYYuNmEOUgQWiUgdJqS_TIciLCatXlXcreV9VV7YI9u14IdrCKp5bqz1McGMO3QcM-TwIjTuCTz0phziy1ODOZnguLqK7Vu95Yg_tJ0xVE-zB2dT-48OAY2Nycc7TABsz-bur5PPPns6YCZRvN9Ubt5H2fvWu40ACEsw7yxBS6D4tHOZPw6tFeaTfw4CngFXrTPktrVKfGLhiSK7FRFZqG7VJ3zcdE79VAworQSBk-WLfgCiiFd-1G00

Plant UML link for the aisle device:
https://editor.plantuml.com/uml/ZLLjRzCm4FvUgV_Xj0_GiPN1U5M59ifMfagiZAq844EHczmIQubTxCI6WFrtpaxTD5q6v4FgFdT-xixFNNmVffcqS9mBHV3nw1qaKgH659092pD41GRFHKpQO0929H1dGgMypb1SolmQPjSm73_iThgTrvQ4eRB8OIXjZWTuAMDq1diXrLmuqZWNrWw9Z9oLXEDOgrFv4-7HBH_urUq0Fsz4V94QNQfaN-VQmATC4YvCsYHeAlXDpduLF3H6NpNG6vT3nRgIsBvMP7IUemcn-Z0J5gFIv300IoQWHiyqCNJYVWD8ii384ixP8DKI5Yv099CcGwxZ2uxZCVWD4wqmcBwNM29RScdfQo9Z-XOidZsAkNPgnEXNlrjWKILHWTQA57i-IGxiOsn3OLE_sWpEvJmHv3BcTs-9cbAvFoidv8jiEvkuD0OLHRwM9FA5jbPK7O51EzVAjheiAs0lhjzInHOnlRdkAPrWf4I1Yr92O8StQsA8vFbrsFr3KbrPDEVycJz6UObclSW53AzUPS8aLyBWtbueTGjGYLcElQN4_lINp2dI1oVQErGfPRtDLN6yzPJRMwNUiEo1BL3uWuwv7t4bINRUvt5IcDSqAscE2abqPvPE_XwBAWOyWuIsMU9RVb-uwM6lMiJgf7mJM0xe-crTa-FtfrFO4NEvuvrtl7Cm7evks-mYBek1O6IQ4UZpzabpD9-G6ynohRnPnMW73knN_j5Zw5MBPz6BgDgzkMhV3VRTDkkJslgqpVuqs9z7BskxEo2N-x86pezqyTSYggdr_Jg0Nn26RnjSay9DatJEAl28miPHDQjmR3JfMESRGPXAih6ATS6Tit2ph3kgia0ZI6fLI--Uaz67ZwFJQJIQJ4--rF3eytWwdXnMmCqg-VkIKlsVv9uqEZmPJgQZWtNuO3GP1_Hc8VsjpFjVCvaagBub-ahrzzQ_AjtEhNJMTznnzDrE-oVyGT_IgoTx067s_xKbJDa0i5gvAEkVmRvlxBloSfFEYjkcB6VgNX04BgMewTivbY7MOmVoXYndQgSXo7z5M9njuzimQ9_uCzKBTzdsulxQXYiXYKNRzBSzR-Qx_Wy0

Plant UML for the frontend:
https://editor.plantuml.com/uml/ZLNDJjj04Bv7oZkCu08jIMXxYuH4Q0EAIbkAKJcYZNUmLx5t&tTqrKQYuzW7wY7sIpgxZU9t4AiubdfzlPxvlTdnkBDEsp3Eur4fQbBpVw_VCGiY2QPP3d35Z9jPgCIyjpc8b8_6CSBeJ0x_wFQ3dWiMBHAjIyeygKnhkKs5nxLAQewxC5nd5LUQ9rchPihw4w0MPM89T-8UNf_Ixx3pXnVLJOnpKJS9Hz8IPnS_27Xyuw2g_wkIQhLHfwmFTCm3CKISeuzKajk89OGnpfR8WOj3yVmjM9Kc6qphdw3XqnYd6YuqIiioM9emO3kzn7gbuWNOuRCnAZfig7ig2CuiypBgbej64AIFu-_iFV2-nn1jgFt08-OJIAhto4RFwRIigPuNtVs55utbfaxL1R_DbKhMyKKLPx3854fSpY_cCayzO7ReqMth0JymoyZAjsIhiaTGbzddOv24mBudFenmgka0EF_qhpLvkkbJI-AZHf3wnBL7A9CymAkUviA-KnfzNo-9Pwu8V3gyLQxLmDRs34Il4o8ECZ1-9ndtpBGhzUsLeITYMmdEmIoPfX4p7wHtgV5k83gARgTdGl4Nm8JmAkbOQ-Io8hMqmNu5aET9W2luVLJRvh-H_Kzi-3Jh0EdK8kjhYUNqLsWmJ5-k1bOgZeIQG9nZE7J1XQA18M7HIS17RbaJ6bI9aikUsPEIPqehKko9rak5FsxSZEZkc5lPOFOwVnx0RMZMqGcb0gDwZwCVLSOZurJ5m0bD7m0dSghco3PEUpFLwVJFWQYa7mG9npnX6En7jsHu7ssidj9O7QZ3VLxydbKNG8aajgCTmdrNVYbXvyhMlTkpi5ye-I5kF4AyDxxrXdbLtq5a-U4kY4QKpJHnVNlnnPJoZySCJXt6cuhWiX0DptnzNa2lid5xewzll_GC0

Plant UML for the backend:
https://editor.plantuml.com/uml/XLLHRzem47v7uZzOsSCUAahtYYfKkiA4jAsTgDJ7o2Idp2MsCzlfn3R--yus8OuJIdY1-yxVVVVbpjnfGvMfUKdkQRO3aOz7uv7UCL5HHJd9IghrmXZ5jhM1TIR5Xlq1SjFB8N_78uAFPIcKh4N-MPPIaPUVpC0HaYe7vSFt9URvy48f-RiJFOJi5OPe0UVuFwnkyBCwM-5W-sauaXLAtJMT9fRJ7_GjE7Jnj6vgsMT2IedPUIfa3fhCICuoCm1Jfij1t04fkcm6OU2W2X3PFgMPOM-0ALifoo1bqdx_YB_8Mho1C5BjEz0L-R9y9bDQiIaQpdM8F3rkZj2I6cg8-9wj9Jyi4156nwPe4kFI9aXUbM1WK9LFiJxKEbAmi1QHhsWa4yK54SxEgJNzdIfj4kINoMeHqmtdAT1WKbgMaVpIE1aNrRlnwUcEyRwKKsSjGTYJ4_2DLkTcWTDgqEoDdSUpOds_RcO7MGCyVfVBPfxFAJeD_73f23vqDtHZspVhnwLT0KrkRq7KU80Q9iLy7kwjUt5fNUMeCV2BDSIJX_oYziBjWdQWGtorF513jrH3xrPnwnXAko8inpLdmeGnGRbTVcrK6CsmOc5tlOTeWrxqQLmNOJXoy5LkK-VDPKLMU3e4-2N6U8Mtg1twVmIfMI4Wnw_DBT3PF2MtqdWox0EpH1tk4JQgm9gRKeCfXd708hpgUdswdoUJ-T5M_ym8diurcHB_ZhHt9phHjcvF-8BTC_vkEXswlfx74pV3UM16MpKjNSAfs2UkHI8fEIgRDVxPt3iq2lzZnwF_

Plant UML link for whole project:
https://editor.plantuml.com/uml/rHfTSzisyjqp_WyOTwQH6ijEchiNJPgfR2k9flwgfCJjn1aEHC8I6X9G0SYkBI5_lmL0KW09qiv5UJZvGJ9tiH_OJopn14qNLABB53E5tfsTeeJYkS0PMc29PeGm9CWDZXKN946O9IXUO3Q7tmk2RscwHhCr6emETdTsTtwL2Wkrob9qXGLPy9KaaxLK9DD0-Oco9TQ4unHBEL1AqDbAaL7CsOJ-Xw1dDHpqUNS7mUS8nv_cWgzOSinJBj3LWYgIWxX8YB2FZrB0iuy7Gl0xx-czltq9Z_2SDD7_uVKp-7lToE5Uw_C4_V8D7xt-zVZYV3eyF_bsMhixXKPxhmLdYe2PEfDRaYho6rNTFQserSRkwobUyvKgrDUV7Y8P4NF2ulKWLIIMe3wQSPuw6BtDxwT8yVay9SDYJQVh0kC5YJ-Li8d2QYLTZ8E3ApARyFWJKGS7cyUSzJTIHAjbWXL9BFZUb__t5LcHIyU2LWM1xmpa12iCycCXyDg5Ct8t0cyS9G2KIZJf1lukY5mOKhvYuFb9IYQhMKRL8nNwMrDfLMR4RWc3y5hN5G8tpsH84qcmY1TJ8h97gLAoUAJGjC1I5VmCB-iYPtYPRp_Z2P4W0qdcHBeec4hOHF0AedLBQAmU9PHPrY1Gxj0_zH9-ntgEV-XF7nq6iVoD1wnIxI1weIugY8AgMyi5HuFZtxQU2i1AdqnoWtnGom46EhWS-UPBzTOdaRLTrJWbs9WY1BSsac7REZacmez9Cc-6KtI8b_JGn84BkRoOv20JNosWGz2MPgq8F5kcH94WPuiYJGfh8s8KFzJRqq9cWrGuS1kn5QkI2-D1W1CLuJGDenfJrYIhmkICIjat17ofImGJ1o7ILZxZ3S653bm64Kf_2TEIW2IfoMQHqSLG1xL4BryIje85M57ENhrotJwGe_Md8GlgJo0NwSz3rQkCkOE3LrNvTEsZIcujevmCfeEZmMIurPIYK_iCIrBB9hPIVNRjH7MPfKovpnZEIBLYGRy0MpM7VQv3f89DgfCns-y-hfZ-BpwBZ54VbaWB7eK0DkRX-HAo9sLpz0K1QJfdqFf-GKNIy292y1bNbXZe0LXivKTbB8ZsY

WWhG54q8y0aMrPKs0Y_bs6steEa-0hjFTyhku5SvArvolZYw6AAZg4P7L-SdWx7M_MOiTuJ
T0nDgU1fIaJDSmp21kwNemLEGM63ydPmwaBqxa6X8SkgvNgv6KAk1iiW065NGJG1m2dy
TsR_yxehdP7gQ-EL412gfZZMeN1-C8jIAjK7W7qCBTNijUbNu3pW7-US4HTjJ33qpibbldcrwe
Zu32YlOkMKw5P-RhqklRBArIOpA2DBpcGpsbSMRx5YRhVPGxOj0CTDIH1mXwc2E0l218U
l9c36Rzj1Llysfa4C1kVRkKPI7F3MImv-rf4fluliUTZ75WI2LYfBAOBDXsvRsyWuuhxdF1KsND
6RTR7k7_za0V0bn0LGZmdikQZ0x84aqcw1Or00pjzpOdtVH-IiL05e6IzhHJRU7DrGHgL6zU7
cV1Tj_47MJdY0f0HcakfYsu5pbzrk7oCHe54d9JTg7mawNy1NVNVl0za2Ke6V9Y8JyaMcyCL
ep1EUjnORqUdMYv26QJ86TfDoh49GmyIWr0e9F2yn0WX0uOJSqfXq4lDbBG9uVFONYQiL
hf9PtsAHt65XAt91q-xLcyi9T3sB7C5lUIndml1CUqqfN1L1AxVakHcxHROnU-3ac_igvsEeZlfx
2zmJ2gx_bUnFHoU3yRSojmYNEcAEKxxoOc0Zdj2pi8XcO96l4F2CWuC_8A1qjZwYJ8UY8d
KX0EsoC12WvmuQGdjFe1fGjUxSrf5g4kuDeHbLK2GxniswUz3ysEWg00z9pv8EjC0OYiGQs
IjMNkCykTMPbP1IzoGp4pWbRAuMzIWjiIPAc5fIpG8MWP5_b2xEnClHzy5C1JiWkT0naF5ab
LRahZftR2Ccc66eZ9POBI98y5XqKffWiO_KGWzsU9fqQtNuAKgWCurrGjDZq2ZcGj24vqj3-5
eMIQ5M9U2-cCL4Rd1Dh-8YswmKCH1cjdv0dCgUP5mhsxWbcu80XHiY92yB-J_Np43aaf2a
WDX_06Abi395fJ2FA2XvaMyJ2-oHH738DLvXQhhSjrEjRdKJJTdqK42uIFJlEzqR05dIhNTIL
HzQSA5qKw7jtoYVXrKmTtXTCoi9CkFvHx9sMkFAaBMvkwvLLDtkHffFXvKSZJzzT36BivH1x
vWrDEdw5e1-mxPejaqgAcJLaUqOm-_cIiMg9usVUhrDFz7NPmq7Jp_J2Bg8zj5yAGEmeiprK
Hw-0HnRYxJkuA41k2q502-z7XswRXzOugLXEpPGf16nJA5zL3IHJHkYfn95skgZbIIo6IyFvO
VGdF1oKPt7XgddvovYpiQNpCj0RTPrdRZldlssTdOUZ2QdGtGoV3yw7dwl8_J0dDviYzXsaZ
Pu5gtfF6sZXzPVWDZ9hcWy_Dc1H1x2UgpxiIdkTUMFCYAbRecAYOWXkuyoEJU_lE2sep9x
GF1VjpX7o3A8uRH4R7mWVXDMkI1nHO2vX4AVrxH-0J2hekVx-O-VYnylYX__AZ2jskOmrq
UVom7FldcvKHH-_RPDYrNhq3t3jJJgcsRBhB3zZ_dvNPkjAhF_YsQhN7mCPrz6qZhvQZiM3
WkT7hmjhYJI1rO5i-fMvt6l9sVVOH07L6jvOtG2AUpbInE0li6DuqCzqYC8FttMC5BCyX7OBS
LEBxI9tmt3gy5eEZf_qyuJJYn8JsUY5LCqDVC-Xs_20SKKIAc7n4lKURQFdeMOZI-cWsPkXe
uWE365_gbVo3S8t4m4hIGuDKFW0AkpY_VjZ48JfMjcuEvW2Vs2NeHKiVGZMtEsmUNvmR
-xbSKwbw3X7zP0K7djw52FfPeDsIRHnfn0Tppy_TrmCes6K2V_VAGnImJZXECosBmODovS
VlJ0l3Ix-VHptxpHbpoQbXHkMInXF5_ruSKpQHtmOzDQ_ubPQsQz7-0imfM2BVnPvejzDg75
XXNi-HAEXxcqlZfwKUvivQBSBT6FFrupEzdJHSdQhvr6Rhum3Ifp5z3FSAf3PGtvjfskxnSFqz
M2hlL3ZwwlOSYenTmpF_2Iv9fr1356wpwMHX4XA1-Bmd4fuuAKpoK0CanP4xsgyN97ooUd
xgfYdn-hYHUA-Yv7NR4sal-BOYwznoZc6phaKYRKepVZmVbqU5BKokszkpeRJYQ3DqDq2
TfV75-SRhLadjak5br22FEOfsNhvioYfznsUlslbnkoNcxPG5er5AVSLkmMmjxULX-V3Cz7nTD
xvuXK2DpxiA19GjX7qqptFAPRCy_xuVXeT7u2QWxFqVJTP09Vapydq-7Py7mRfdg2oljkBy7
vyhikgFDsEhs4a2dl4a7G-7U2z86oV1lV3z_tk6QrQoR-bQ4c8Ibr5V_2K1DoXfVNhE5AH7u3g
P72b-1TYDsTojK2lT0d5Rne5rXLlKXLkIaHM56zL1MuH15OLRrRLRbVOMv8vRRLTYwk8Up
kbFkXSuzzGMwJ5TGf2Ks39l7jHO8A-kP5Upz_C--li7T-J4Tk8x7gOU1LxeIXOf_WHQIQJvNN
Z-gGpNMX9bXnQoSqUGdvckSaJzmpJb2vy1ceKKtlke-_RlwdpXMqDIcarcvTy9vDHgyQZbzz
FvU2jV8a2jLI7ploiroZw3qzPTEbiiAsLgSEIIHQexwxyoliqofBzS__0W00

Annex B: Plant UML (diagrams)

Plant UML link for FSM states and changes (state diagram):
https://editor.plantuml.com/uml/nLRjRjis5FqlGEzmWG2PxIPRkcBeO215tKK93AHEQxlhXgGG6EdQ9YoH0aaxCuO1UOW-OPzabnHbIx6JTVtX2eOXNehd7fxxGRxMXYcpo3COobjfpaTlmo0Cz9oBWYcM0qqRXB_30EXvmvBvLCc5I7-JcLJmSSOD-YcfKbIb-Kr6tvNcdbBoRiiwiZwth545wst_r7a8-JYDW_FpyxFevJQHWr_Tuw32uBhp2OwFNq7_x39octHRiwCAsBrt0OLHA_W92fbbO9YUvqomAQeRKMHC07ccTKlEO8ecTgDMs_ieCJ_s-kF-uAA2hOORP8q6xXWtyK8OJZvuRj5JILEvD59fqbqMq3evWfCczF1gtAjZhyTTSCiLiXGktetW6lGoBvZM7ZTLV8cj9SkEu9Z-sr2wPXcV2amRFjvU_Lxpu4SUNx0bdz97iP7n7LiLaWlJqfcyYoSi8UQdBzhUeIMHIAKmCLmAsX2YucBgPo_xPxqXQK0ArpxYmeruAjKEHd6lFze4gs7xMdRFV_obugTHf13zKOQ6V3EkCmJyYvjTiMpwRXhhSINCOVJ-GpGQnz5WFFpJvKcKSsEUYB2VSFasn0GfKWecbC0ATI65Hi_e1_Zo-Pv-PQ8QnOJcLZVjxVlyXO4ZOGkgoc__TAbOVP5CR97UY6kRw_3YH7-ghLfdRhMgjgJscNlPF0xSrYIm9UCPkysGyCrCIMCeV3zR5pMXAchVHQimgDm_t4YtB7dAje8hMtqrocjTziyv3DREBMDV-HIIIbd0uU6De1PExS_svR9oTgxr1VB8MgvzqLaRosphMjaMi0kIMN5VWMU9hQnnZLVJsE_iUrLG67W2MtNG1NbBVIUXJQHeg2iYbKzBOLaRHaAoe6OfhEufjf_2gTgVFH6IcHSgbmhNCvgcSiP50wLHRkkQv8BhcQDIy6I-ADnv8WivcG1X9_Fs3dLzgFQZSRK7x_I17hjLRHm0py2BtFulgFylR0sf9kytYjiiX3sek-vYjMqyE7ITIHTV37k3SNHs8rHvX3NRMaJzReeYMO6aqusEUDnpoRaR9DCR8WDPNbuxdMrkdGwqdFWKNoJRojvVYYEunOcDT7arc36HPgZQ5gG_0QnGOhfhqFxXz1J6Qe7TyiwmVUp3b_lF7efQr-1gJF5cOcg5iwaf7BymU4q3kg27mRy0

Plant UML deployment diagram:
https://editor.plantuml.com/uml/TO-_3e8m4CPt4zxXmgHl0SZYPdHmlfOR6aklQGiEndUt56eGOzAXz-VxlljLFg0BGw-BdU6Ee6Lv9mVFflpR1YIRWCgGAwUsvDwo8HEWnO02FHMxLzw-aV3ancZmJrJMrcebCIWsiB-Ef0CTla0FQEttula40kCrfalpvP_ISc3Z-E7dSrVAsHLYKXBwL2PHXgHHIV8VoWiB3XkJkNUC68wrdZYYbOjOAznQUPtTVZYdsJpQkc2NRUAh8jFGlm40